

Preemptible I/O Scheduling of Garbage Collection for Solid State Drives

Junghee Lee, Youngjae Kim, Galen M. Shipman, Sarp Oral, and Jongman Kim

Abstract—Unlike hard disks, flash devices use out-of-place updates operations and require a garbage collection (GC) process to reclaim invalid pages to create free blocks. This GC process is a major cause of performance degradation when running concurrently with other I/O operations as internal bandwidth is consumed to reclaim these invalid pages. The invocation of the GC process is generally governed by a low watermark on free blocks and other internal device metrics that different workloads meet at different intervals. This results in an I/O performance that is highly dependent on workload characteristics. In this paper, we examine the GC process and propose a semipreemptible GC (PGC) scheme that allows GC processing to be preempted while pending I/O requests in the queue are serviced. Moreover, we further enhance flash performance by pipelining internal GC operations and merge them with pending I/O requests whenever possible. Our experimental evaluation of this semi-PGC scheme with realistic workloads demonstrates both improved performance and reduced performance variability. Write-dominant workloads show up to a 66.56% improvement in average response time with a 83.30% reduced variance in response time compared to the non-PGC scheme. In addition, we explore opportunities of a new NAND flash device that supports suspend/resume commands for read, write, and erase operations for fully PGC (F-PGC). Our experiments with an F-PGC enabled flash device show that request response time can be improved by up to 14.57% compared to semi-PGC.

Index Terms—Flash memory, garbage collection (GC), I/O scheduling, preemptive I/O, solid-state drives (SSDs), storage systems.

I. INTRODUCTION

HARD disk drives (HDD) are the primary storage media for large-scale storage systems and have been for a few decades. Recently, NAND flash memory-based solid-state drives (SSD) have become more prevalent in the storage marketplace, due to advancements in semiconductor technology. Unlike HDDs, SSDs do not have mechanically moving parts. SSDs offer several advantages over HDDs, such

as lower access latency, higher resilience to external shock and vibration, and lower power consumption which results in lower operating temperatures. Other benefits include lighter weight and flexible designs in terms of device packaging. Moreover, recent reductions in cost (in terms of dollar per GB) have accelerated the adoption of SSDs in a wide range of application areas from consumer electronic devices to enterprise-scale storage systems.

One interesting feature of flash technology is the restriction of write locations. The target address for a write operation should be empty [1], [15]. When the target address is not empty, the invalid contents must be erased for the write operation to succeed. Erase operations in NAND flash are nearly an order of magnitude slower than write operations. Therefore, flash-based SSDs use out-of-place writes unlike in-place writes on HDDs. To reclaim stale pages and to create space for writes, SSDs use a garbage collection (GC) process. The GC process is a time-consuming task since it copies nonstale pages in blocks into the free storage pool and then erases the blocks that do not store valid data. A block erase operation takes approximately 1–2 ms [1]. Considering that valid pages in the victim blocks (to be erased) need to be copied and then erased, GC overhead can be quite significant.

GC can be executed when there is sufficient idle time (i.e., no incoming I/O requests to SSDs) with no impact to device performance. Unfortunately, the prediction of idle times in I/O workloads is challenging and some workloads may not have sufficiently long idle times. In a number of workloads, incoming requests may be bursty and an idle time can not be effectively predicted. Under this scenario the queue-waiting time of incoming requests will increase. Server-centric enterprise data center and high-performance computing (HPC) environment workloads often have bursts of requests with low interarrival time [15], [22]. Examples of enterprise workloads that exhibit this behavior include online-transaction processing applications, such as OLTP and OLAP [6], [24]. Furthermore, it has been found that HPC file systems are stressed with write requests of frequent and periodic checkpointing and journaling operations [31]. In our study of HPC I/O workload characterization at Oak Ridge Leadership Computing Facility (OLCF), we observed that the bandwidth distributions are heavily long-tailed and write requests occupy more than 50% of workloads [22].

In this paper, we propose a semipreemptible GC (PGC) scheme that enables the SSDs to provide sustainable bandwidths in the presence of these heavily bursty and write-

Manuscript received February 13, 2012; revised July 29, 2012 and September 18, 2012; accepted October 22, 2012. Date of current version January 18, 2013. This work used resources of the Oak Ridge Leadership Computing Facility, National Center for Computational Sciences, Oak Ridge National Laboratory, which was supported by the Office of Science of the Department of Energy under Contract DE-AC05-00OR22725. This work was supported in part by the Korean Ministry of Knowledge Economy under Grant 10037244. This paper was recommended by Associate Editor J. Henkel. (Corresponding author: Y. Kim).

J. Lee and J. Kim are with the School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA 30332 USA (e-mail: jlee36@ece.gatech.edu; jkim@ece.gatech.edu).

Y. Kim, G. M. Shipman, and S. Oral are with the Oak Ridge National Laboratory, Oak Ridge, TN 37831 USA (e-mail: kimy1@ornl.gov; gshipman@ornl.gov; oralhs@ornl.gov).

Digital Object Identifier 10.1109/TCAD.2012.2227479

dominant workloads. We show that the PGC can achieve higher bandwidth over the non-PGC scheme by allowing preemption of an ongoing GC process to service incoming requests. While our previous work [26] discusses only semi-PGC, this paper also demonstrates the feasibility of fully-PGC (F-PGC) that supports suspend/resume commands for read, write, and erase operations.

This paper makes the following contributions.

- 1) We empirically observe the GC related performance degradation on commercially off-the-shelf (COTS) SSDs for bursty write-dominant workloads. Based on our observations, we propose a novel semi-PGC scheme for SSDs.
- 2) We identify preemption points that can minimize the preemption overhead. We use a state diagram to define each state and state transitions that result in preemption points. For experimentation, we enhance the existing Microsoft Research (MSR)'s SSD simulator [1] to support our PGC algorithm. We show an improvement of up to 66.56% in average response time for overall realistic applications.
- 3) We investigate further I/O optimizations to enhance the performance of SSDs with PGC by merging incoming I/O requests with internal GC I/O requests and pipelining these resulting merged requests. The idea behind this technique is to merge internal GC I/O operations with I/O operations pending in the queue. The pipelining technique inserts the incoming requests into GC operations to reduce the performance impact of the GC process. Using these techniques we can further improve the performance of SSDs with PGC enabled by up to 13.69% for the Cello benchmark.
- 4) We conduct a comprehensive study with synthetic traces by varying I/O patterns (such as request size, interarrival times, sequentiality of consecutive requests, read and write ratio, etc.) We present results of a realistic study with enterprise-scale server and HPC workloads. Our evaluations with PGC enabled SSD demonstrate up to a 66.56% improvement in average I/O response time and an 83.30% reduction in response time variability.
- 5) We discuss the feasibility of F-PGC. When the suspend/resume commands are only allowed for the erase operation, the average response time is improved by up to 8.00% compared to PGC. When they are supported or read, write, and erase operations, the average response time is improved by up to 14.57%.

II. BACKGROUND AND MOTIVATION

Unlike rotating media (HDD) and volatile memories (DRAM) which only need read and write operations, flash memory-based storage devices require an erase operation [29]. Erase operations are performed at the granularity of a block which is composed of multiple pages. A page is the granularity at which reads and writes are performed. Each page on flash can be in one of three different states: 1) *valid*, 2) *invalid*, and 3) *free/erased*. When no data have been written to a page, it is in the erased state. A write can be done only to an erased page, changing its state to valid. Erase operations (on average 1–2

ms) are significantly slower than reads or writes. Therefore, out-of-place writes (as opposed to in-place writes in HDDs) are performed to existing free pages along with marking the page storing the previous version invalid. Additionally, write latency can be higher than the read latency by up to a factor 10. The lifetime of flash memory is limited by the number of erase operations on its cells. Each memory cell typically has a lifetime of 10^3 – 10^9 erase operations [14]. Wear-leveling techniques are used to delay the wear-out of the first flash block by spreading erases evenly across the blocks [8], [19].

The flash translation layer (FTL) is a software layer that translates logical addresses from the file system into physical addresses on a flash device. The FTL helps in emulating flash as a normal block device by performing out-of-place updates thereby hiding the erase operations in flash. The FTL mapping table is stored in a small, fast working memory. FTLs can be implemented at different granularities in terms of the size of a single entry capturing and address space in the mapping table. Many FTL schemes [11], [20], [27], [28] and their improvement by write-buffering [21] have been studied. A recent page-based FTL scheme called DFTL [15] utilizes temporal locality in workloads to overcome the shortcomings of the regular page-based scheme by storing only a subset of mappings (those likely to be accessed) on the limited working memory and storing the remainder on the flash device itself.

Due to out-of-place updates, flash devices must clean stale data for providing free space (similar to a log-structured file system [35]). This cleaning process is known as GC. During an ongoing GC process incoming requests are delayed until the completion of the GC when their target is the same flash chip that is busy with GC. Current generation SSDs use a variety of different algorithms and policies for GC that are vendor specific. It has been empirically observed that GC activity is directly correlated with the frequency of write operations, amount of data written, and/or the free space on the SSD [9]. The GC process can significantly impede both read and write performance, increasing queuing delay.

A. Motivation

In order to empirically observe the effect of GC on the service times of incoming I/O requests, we conducted block-level I/O performance tests with various SSDs. Table I shows their detail specifications. We selected the Super Talent 128 GB SSD [38] as a representative of multilevel cell (MLC) SSDs and the Intel 64 GB SSD [18] as a representative of single-level cell (SLC) SSDs. We denote the SuperTalent MLC, and Intel SLC devices as SSD(A) and SSD(B) in the remainder of this paper, respectively. All experiments were performed on a single server with 24 GB of RAM and an Intel Xeon Quad Core 2.93 GHz CPU [17], running Linux (Lustre-patched 2.6.18-128 kernel). The *noop* I/O scheduler with FIFO queuing was used [33].

To measure the I/O performance, we use a benchmark that exploits the *libaio* asynchronous I/O library on Linux. Libaio provides an interface that can submit one or more I/O requests in one system call *iosubmit()* without waiting for I/O completion. It can also perform reads and writes on raw block devices. We used the direct I/O interface to bypass the I/O

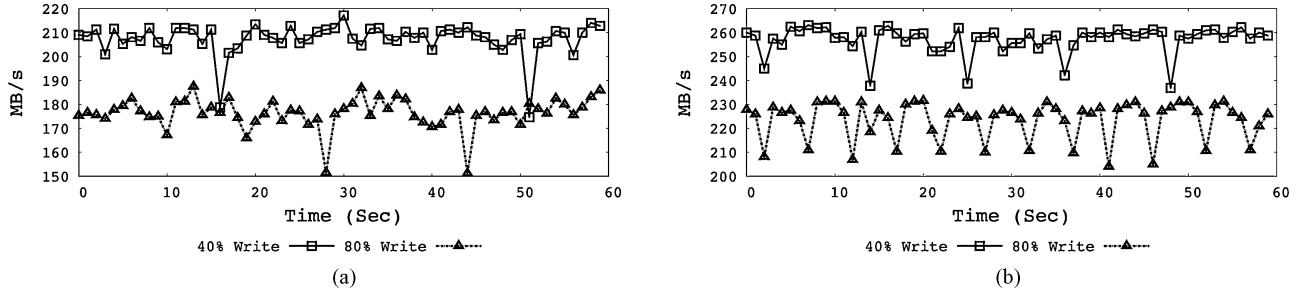


Fig. 1. Bandwidth variability comparison for MLC and SSD SSDs for different write percentages of workloads. (a) Write-dominant (80% write). (b) Read-dominant (20% write).

TABLE I
CHARACTERISTICS OF SSDS USED IN OUR EXPERIMENTS

Label	SSD (A)	SSD (B)
Company	Super-Talent	Intel
Model	FTM28GX25H	SSDSA2SH064G101
Type	MLC	SLC
Interface	SATA-II	SATA-II
Capacity (GB)	120	64
Erase (#)	10–100K	100K–1M

TABLE II
AVERAGE, STANDARD DEVIATION, AND CV VALUES
FOR FIG. 1(A) AND (B)

Type	Metric	Write (%) in Workload	
		80	40
SSD(A)	(avg, stddev) CV	(176.4, 6.37) 0.03599	(207.4, 6.73) 0.03249
SSB(B)	(avg, stddev) CV	(223.5, 7.96) 0.03561	(257.1, 5.86) 0.02279

buffer cache of the OS by setting the *O-DIRECT* and *O-SYNC* flags in the file *open()* call.

We experimented with two workloads of 40% and 80% writes. The I/O request size was fixed at 512 kB, and request access patterns were completely random. We measured bandwidth every second. Fig. 1(a) and (b) shows time-series plots of our bandwidth measurements for SSD(A) and (B). We observe that: 1) several bandwidth drops occur over time for all experiments, and 2) the bandwidth drops are more frequent for the workloads with a higher amount of writes. In order to fairly compare the bandwidth variability for different workloads, we calculated coefficient of variation (CV)¹ values for each experiment.

Table II compares the CV values for the experiments. We see that a higher write percentage in the workload shows higher CV values, which means higher bandwidth variability. We suspect that this performance variability is attributable to the GC process. This insight led to our design and development of a PGC. The basic idea of the proposed technique is to service an incoming request even while GC is running.

III. PGC

A. Semi-PGC

Fig. 2 shows a typical GC process. Once a victim block is selected during GC, all the valid pages in that block are moved into an empty block and the victim block is erased. A moving operation of a valid page can be broken down to *page read*, *data transfer*, *page write*, and *metadata update* operations. If

¹Coefficient of variation (C_v) is a normalized measure of dispersion of a probability distribution, that is, $C_v = \frac{\sigma}{\mu}$.

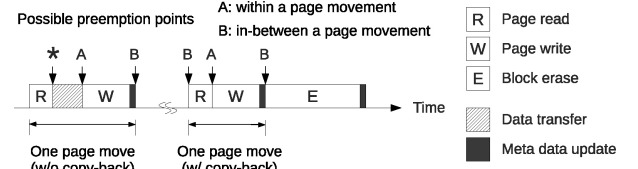


Fig. 2. Description of operation sequence during GC.

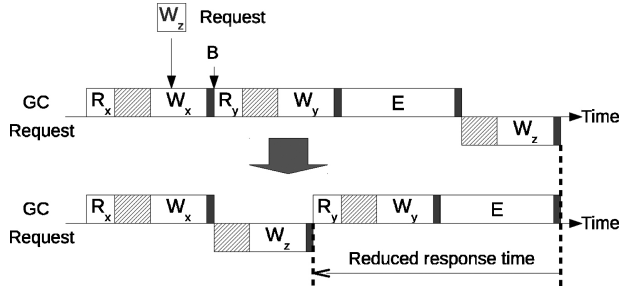


Fig. 3. Semipreemption. R, W, and E denote read, write, and erase operations, respectively. The subscripts indicate the page number accessed.

both the victim and the empty block are in the same plane, the data transfer operation can be omitted by using a copy-back operation [1] if the flash device support this operation.

We identify two possible preemption points in the GC sequence marked as “A” and “B” in Fig. 2. Preemption point “A” is within a page movement and “B” is in-between page movement. Preemption point “A” is just before a page is written and “B” is just before a new page movement begins. We may also allow preemption at the point marked with a (*), but the resulting operations are the same as those of “A” as long as the preemption during data transfer stage is not allowed. At preemption point “A,” only a write request can be serviced if the NAND flash memory supports pipelining commands of the same type because the page buffers are already occupied by the previous read page operation. The pipelining will be described in more detail in Section III-C. If the NAND flash does not support pipelining, no request can be serviced at preemption point “A.” In contrast, preemption point “B” can service any kind of incoming request.

Fig. 3 illustrates our proposed semipreemption scheme. The subscripts of R and W indicate the page number accessed. Suppose that a write request on page *z* arrives while writing page *x* during GC. With a conventional non-PGC, the request should be serviced after GC is finished, as illustrated in the upper diagram of Fig. 3. If GC is fully preemptible, the incoming request may be serviced immediately. To do

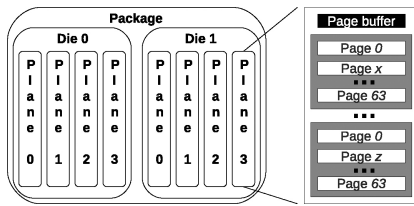


Fig. 4. Internal structure of NAND Flash device.

so, the ongoing writing process on x should be canceled or suspended first. However, there is no NAND flash memory so far that allows ongoing read/write operations to be canceled or suspended, to our best knowledge. The F-PGC is discussed in more detail in Section IV. In PGC, the preemption occurs only at preemption points. As shown in the bottom of Fig. 3, the incoming request on page z is inserted at preemption point “B.” As a result, the response time of writing page z is substantially reduced.

1) *Space Overhead Discussion:* Our proposed semipreemption does not require an additional buffer to service incoming requests while GC is running because it exploits the page buffer that already exists in the flash device. Fig. 4 shows the internal structure of a typical NAND flash device. One device consists of multiple dies, each of which contains multiple planes. Each plane has a page buffer and number of blocks. The pages in the block cannot be directly accessed. To read data from a page, the data should be copied to the page buffer and read from that page buffer. Data should be written through the page buffer in a similar manner.

To move page x in GC, the data on page x should be copied to the page buffer in the plane where page x is located. Then, the data should be moved to a page buffer where a free block is located, and then written onto a page in the free block. At preemption point “B” the page buffers are available in both planes. Therefore, to service read and write requests on any page, the service can be launched through the page buffer. In contrast, at preemption point “A” the page buffer is already occupied by the data of page x . If the incoming request is on the same plane as x , it cannot be serviced because the page buffer is not available. Only if the flash device supports pipelining, and the incoming request is a write request, can the request be serviced. For example, data of the incoming write request can be written to the page buffer while data in the page buffers are being written to a page in the free block.

2) *Computation Overhead Discussion:* Our proposed semipreemption does not require an interrupt. Due to the small number of preemption points, it can be implemented by a polling mechanism. At every preemption point, the GC process looks up the request queue. This may involve a function call, a small number of memory accesses to look up the queue, and a small number of conditional branches. Assuming 20 instructions and 5 memory access per looking up, 10 ns per instruction (100 MHz), 80 ns per memory access, the look-up operation takes 600 ns. One page move involves at least one page read which takes 25 μ s and one page write which takes 200 μ s [1]. Since there are two preemption points per one page move, the overhead of looking up the queue per one page move can be estimated as 1.2μ s/225 μ s = 0.53%.

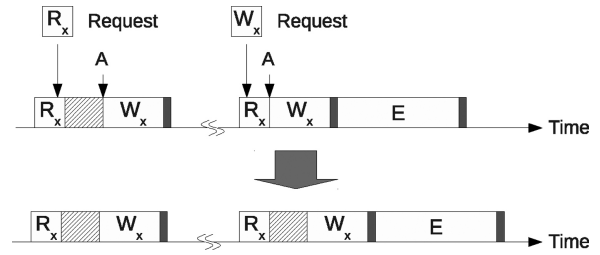


Fig. 5. Merging an incoming request to GC.

To resume GC after servicing the incoming request, the context of GC needs to be stored. The context to be stored at preemption points “A” and “B” is very small because it does not require an additional buffer to service the incoming requests. At preemption point “A,” the block number of the victim block and the page number of the page stored in the page buffer need to be stored in the working memory. At preemption point “B,” only the block number of the victim block needs to be stored. Because the metadata are already updated, the incoming request can be serviced based on the mapping information. Thus, the memory overhead for PGC is negligible.

B. Merging Incoming Requests Into GC

While servicing incoming requests during GC, we can optimize the performance even further. If the incoming request happens to access the same page in which the GC process is attending, it can be merged.

Fig. 5 illustrates a situation where the incoming request of a read or write on page x arrives while page x is being read by the read stage of GC. The read request can be directly serviced from the page buffers and the write request can be merged by updating data in the page buffers. In case of copy-back operations, the data transfer is omitted, but to exploit merging, it cannot be omitted. As for the read request, data in the page buffer should be transferred to service the read request. For the write request, the requested data should be written to the page buffer. We can increase changes of I/O merging operations by reordering the sequence of pages to be moved from the victim block. Suppose page x moves and y and z then, move. During GC, the order of pages to be moved does not matter. Thus, when a request on page z arrives, it can be reordered as z , x , and y .

C. Pipelining Incoming Requests With GC

The response time can be further reduced even if the incoming request is on a different page from valid pages in the victim block to be moved. To achieve this, we take advantage of the internal parallelism of the flash device. Depending on the type of the flash device, internal parallelism and its associated operations can be different. In this paper, we consider pipelining [32] as an example. Pipelining allows overlapping the data transfer and the write operations as illustrated at the bottom of Fig. 6. If two consecutive requests are of the same type, i.e., read after read, or write after write, these two requests can be pipelined.

Fig. 6 illustrates a case where an incoming request is pipelined with GC. As an example, let's assume that there is a

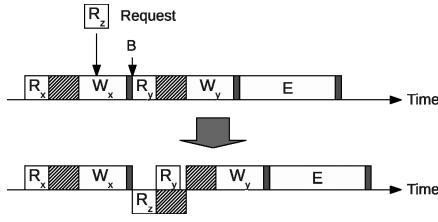


Fig. 6. Pipelining an incoming request with GC.

pending read operation on page z at the preemption point “B” where a page read on page y is about to begin. Since both operations are read, they can be pipelined. However, if the incoming request is a write operation, they cannot be pipelined at preemption point “B” as two operations need to be issued at “B” and they are not of the same type. In this case, the incoming request should be inserted serially as shown in Fig. 3.

It should be noted that pipelining is only an example of exploiting the parallelism of an SSD. An SSD has multiple packages, where each package has multiple dies, and each die has multiple planes. Thus, there are various opportunities to insert an incoming requests into GC as means of exploiting parallelism at different levels. We may interleave servicing requests and moving pages of GC in multiple packages or issue a multiplane command on multiple planes [32]. According to the GC scheme and the type of operations the flash device supports, there are many instances of exploiting parallelism.

D. Level of Allowed Preemption

The drawback of preempting GC is that the completion time can be delayed which may incur a lack of free blocks. If the incoming request does not consume free blocks, it can be serviced without depleting the free block pool. However, there may be a case where the incoming request is a write request whose priority is high but there are not enough free blocks. The incoming requests may be prioritized by the upper layer file system. In such a case, GC should be finished as soon as possible.

Based on these observations, we identify four states of GC.

- 1) State 0 (S_0): GC execution is not allowed.
- 2) State 1 (S_1): GC can be executed but all incoming requests are allowed.
- 3) State 2 (S_2): GC can be executed but all free block consuming incoming requests are prohibited.
- 4) State 3 (S_3): GC can be executed but all incoming requests are prohibited.

Conventional non-PGC has only two states: 0 and 3. Generally, switching from S_0 to S_3 is triggered by threshold or idle time detection. Once the number of free blocks falls below a predefined threshold the state is changed from S_0 to S_1 and from S_1 to S_2 . We call the conventional nonpreemptible threshold as *soft* but in our proposed design the system allows for the number of free blocks to fall below the soft threshold. We define a new threshold, called *hard*, which prevents a system crash by running out of free blocks. Switching from S_2 to S_3 is triggered by the type of incoming requests. If the incoming request is write whose priority is high, it switches to S_3 . The priority should depend on requirements of the system.

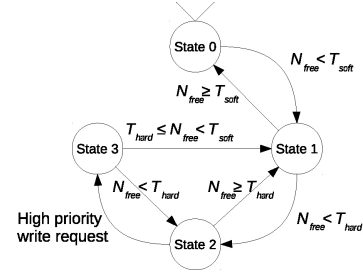


Fig. 7. State diagram of semi-PGC.

Fig. 7 illustrates the state diagram. If the number of free blocks (N_{free}) becomes less than the soft threshold (T_{soft}), the state is changed from 0 to 1. If the free block pool is recovered and N_{free} is larger than T_{soft} , then the system switches back to state 0. If N_{free} is less than the hard threshold (T_{hard}), the system switches to S_2 or remains in S_1 . In state 2, the system will move to S_1 if N_{free} is larger than T_{hard} . If there is an incoming request whose priority is high, the system switches to S_3 . While in S_3 , after completing current GC and servicing the high priority request, the system will switch to S_1 or S_2 according to N_{free} .

IV. FULLY-PGC

In Section III, we have presented a novel semi-PGC with several I/O scheduling algorithms. In this section, we present an F-PGC mechanism by allowing preemption on any ongoing I/O operations.

A. Fully-PGC

A typical NAND flash accesses the NAND flash cells through a page buffer. If a read command is issued, the requested page is copied from the NAND flash cell to the page buffer and the requester reads data from the page buffer. Similarly, to write data to the NAND flash memory, the requester writes data to the page buffer and issues a write command. These commands are used as atomic operations, i.e., if the commands are issued, they cannot be suspended or canceled until they finish. However, the physical operations on NAND flash cells are not atomic. Current implementation of flash operations, such as page read, page write and block erase, has been implemented atomic because the NAND flash interface [30] does not support preemption, however, they can be implemented preemptible. We add a *suspend* command and a *resume* command to the interface to implement F-PGC. AMD’s NAND flash memories [37] used to support suspend/resume commands for the erase operation. The suspend and resume commands should be operable with read and write operations in addition to the erase operation to support F-PGC.

B. Design for Suspend and Resume Commands

The flash operations can be broken-down into multiple phases. Just like the semipreemption of the GC process, the flash operations can be preempted in-between phases. For example, the NAND flash memory usually employs the incremental step pulse programming (ISPP) as its write and erase method because it offers fast write/erase performance coping with process variations [3]. It tries to write/erase by a

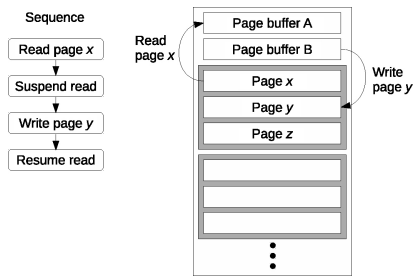


Fig. 8. Example of preempting an ongoing flash operation with the suspend command.

pulse with an initial voltage, e.g., 15 V and then verifies if it is successful. If not, it keeps increasing the voltage by a step, e.g., 0.5 V until it succeeds. Therefore, the write/erase operation consists of repeated pulse and verify phases. In-between phases, it is possible for the operation to be suspended. The suspend command forces the ongoing command to stop its operation until the resume command restarts its operation. While a previously issued command is suspended, a new command may be issued unless the new command is on the same page or block that is occupied by the suspend command.

Fig. 8 gives an example of using suspend/resume commands. For implementing the states of suspension and resumption, an extra page buffer is required. Suppose that a read command is issued on page x . The data in page x are copied to page buffer A. Before the read command finishes, we may issue a suspend command. While the read command is suspended, one can issue a write command on page y . The page y should be different from page x but it can be in the same block of page x . However, if the suspended command is the erase operation, the new command cannot be on any page in that block. The data to be written to page y should be stored in page buffer B. Once the write command finishes, the previous read command that was suspended can resume. Two commands can never suspend at the same time. In this example, write operation can never suspend while read command is suspended. At the cost of additional page buffers, we can allow more commands to be suspended at the same time. However, in order to implement F-PGC, suspending only one command at a time is enough.

If the flash device supports suspend and resume commands but has only one page buffer per plane, servicing incoming requests could be limited according to the availability of the page buffer. For the aforementioned example, when the ongoing read command is suspended, its page buffer is partially occupied. If the incoming write request is on a different plane, it can be serviced immediately, but if it is on the same plane, it should wait until the ongoing read command finishes because the page buffer is not available for servicing the request.

After issuing a command, FTL should check if the command is completed either by polling the status register or by receiving an interrupt. Servicing an interrupt incurs non-negligible overhead because of mode switching. For example, ARM1176 needs 200 cycles per switch and Cortex-A8 needs 1200 cycles per switch [2]. Since checking by an interrupt incurs nonnegligible mode switching overhead to implement F-PGC, a polling mechanism has been implemented.

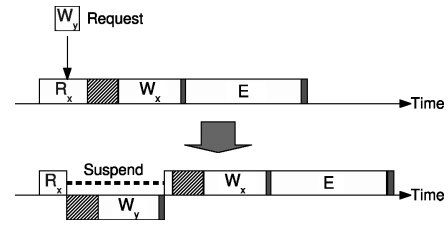


Fig. 9. Operation sequence of F-PGC.

TABLE III
HANDLING REQUESTS ON THE SAME LOGICAL PAGE
OF THE ONGOING COMMAND

Active Operation	Incoming Operation	Action
Read	Read	Request is serviced by the ongoing command.
	Write	Active read operation is discarded.
Write	Read	The request is serviced from the buffer used by the ongoing command.
	Write	Data written by the ongoing command are invalidated.
Erase	Read	Not happen
	Write	Not happen

C. Operation Sequence

A typical GC process consists of a series of page read, data transfer, page write, and metadata update and erase operations as described in Fig. 2. As illustrated in Fig. 9, suppose that a write request arrives during a page read. As discussed in the previous section, FTL checks if the read command is completed by polling the status register. While polling the status register FTL also looks up the incoming request queue to check if any request comes during the ongoing operation. If a request arrives, FTL issues a suspend command to stop the current read command and services the write command. Looking up the request queue does not incur an additional overhead because it occurs while polling the status register and time spent on polling never contributes to the performance.

The incoming request may happen to be on the same logical page of the ongoing command. Table III summarizes cases of conflicts. If the incoming request is a read on the same logical page of the ongoing read command, the ongoing read command does not need to be suspended. Once the current read command finishes, data in the page buffer can be used for servicing the incoming request as well as for the following page write.

The incoming write request may be on the same logical page of the ongoing read command. Then, the data should be written to a different physical page. In this situation, the data read by the ongoing read command are discarded because moving this page is not necessary any more.

Referring to Fig. 8, suppose that the ongoing read command and the incoming write request are on the same logical page and the logical page is mapped to physical page x before the read command is suspended. The ongoing read command on page x is copying data from the NAND cell to page buffer A. When a write request comes on the same logical page, the ongoing read command is suspended. The data to be written are stored in page buffer B and then a write command is issued to physical page y . After the write command finishes the metadata of page x and y should be updated as valid (V)

to invalid (I) and empty (E) to valid (V), respectively, as the mapping of the logical page is changed from physical page x to y . The data in page buffer A were supposed to be written by the following page write in the GC process. However, in this situation, data in page buffer A do not need to be written. The purpose of moving pages by GC is to move and invalidate all the valid pages in the victim block. In the case of page x , it is already invalidated by the incoming request and the up-to-date data are written to a different physical page. Therefore, page x does not need to be written by GC any more.

A request may come during the data transfer. Here, we also assume the data transfer is issued by the CPU. While moving data, the CPU also needs to look up the request queue because we assume an interrupt is not used. If the CPU looks up the queue frequently, it may shorten the response time of the incoming request, but it delays the completion time of the data transfer due to the overhead of the look-up.

When a request arrives during a page write, it can be serviced immediately by suspending the ongoing write command. If the incoming request is a read request on the same logical page, it can be serviced directly from the page buffer without issuing a read command because the up-to-date data are stored in the page buffer which are being written to the NAND cell.

The incoming write may be on the same logical page of the ongoing write command. Then, the page written by the ongoing write command is invalidated immediately after the command is completed. This situation is very similar to the example of Fig. 9. Suppose that GC issues a write command to physical page x for moving a logical page. Before the write command is completed, a write request arrives on the same logical page. The incoming write request writes data to physical page y , which is the latest data. When resuming, the ongoing page write to physical page x is completed but data in page x are stale. Therefore, physical page x is marked as invalid right after the ongoing write finishes.

During metadata update the CPU needs to look up the request queue occasionally to service the incoming requests. How frequently the CPU should look up the queue also needs to be determined based on the tradeoff between the response time of incoming requests and the overhead of the look-up.

If a request comes during an erase operation, it can be also serviced immediately by suspending the erase command. In this case, the incoming request cannot be on a page in the victim block that is being processed by the erase command. Before issuing the erase command, FTL should have moved all the valid pages, and the victim block contains only invalid pages. Therefore, there is no reason to read a page from the victim block. A page in that block cannot be written because the block is not erased yet.

D. Worst-Case Execution Time Analysis

While SSDs offer better average response time than HDDs, they often suffer from performance variability. From the view point of the file system, it looks nondeterministic when the request experiences long latency because it has no idea when GC delays the request. As will be demonstrated by the experiments, the proposed PGC schemes attenuate the performance variability by reducing the worst-case response

TABLE IV
TERMINOLOGY FOR WCET ANALYSIS

Symbol	Definition
T_{er}	Time to erase a block
$T_{suspend}$	Time to suspend an ongoing command
$U(e_r)$	Upper bound of time to read a page
$U(e_w)$	Upper bound of time to write a page

TABLE V
WCET COMPARISON

Technique	Worst-Case Execution (Response) Time
GFTL [10]	$T_{er} + \max\{U(e_r), U(e_w)\}$
RFTL [36]	$\max\{T_{er} + U(e_w), U(e_r)\}$
PGC	$T_{er} + \max\{U(e_r), U(e_w)\}$
FPGC	$T_{suspend} + \max\{U(e_r), U(e_w)\}$

time. This section provides analysis on the worst-case response time to understand how the proposed GC schemes reduce the worst-case response time and performance variability.

To keep consistent with the previous literature [10], [34], we use the same terminology. The worst-case execution time (WCET) refers to the worst-case response time of incoming requests. Table IV summarizes the terminology used for WCET analysis.

T_{er} denotes the time to erase a block. It corresponds to the time taken to complete an erase command on the NAND flash chip. $T_{suspend}$ means the time to suspend an ongoing command. Since suspending an erase command takes $20 \mu s$ [37], we assume suspending all the commands takes $20 \mu s$. $U(e_r)$ and $U(e_w)$ denote the upper bound of time to read or write a page. These values vary with how the FTL manages the metadata.

Table V compares WCET of various techniques. It should be noted that WCET of PGC and FPGC is of *state 1* where all incoming requests are allowed to preempt GC. If the state is changed from 1 to 2 or 3 due to lack of free blocks, WCET would be increased. Since previous works [10], [34] do not take this pathological behavior into consideration, we only present WCET of *state 1* in our comparison. WCET of PGC is the same with that of GFTL [10]. In PGC, ongoing flash commands cannot be preempted. The longest command is the erase command. In the worst case, the request should wait for the erase command to finish, which takes T_{er} . After it finishes, can the request be serviced which takes $U(e_r)$ or $U(e_w)$. Since the erase command cannot be merged with the request nor pipelined, the merging and pipelining cannot help to reduce WCET.

When FPGC is employed, any ongoing command can be preempted, which takes $T_{suspend}$. Since $T_{suspend}$ is much smaller than T_{er} , WCET of FPGC is substantially shorter than PGC and other related techniques. PGC also offers WCET comparable to existing real-time FTLs [10], [34].

V. EXPERIMENTAL RESULTS

A. Experimental Setup

We evaluate the performance of the PGC scheme using MSR SSD simulator [1]. MSR SSD simulator is event-driven and based on the Disksim 4.0 [4] simulator. MSR SSD simulator has been used in several SSD related researches [32],

TABLE VI
PARAMETERS OF SSD MODEL

Parameter	Value	Parameter	Value
Reserved free blocks	15%	Blocks per plane	2048
Minimum free blocks	5%	Pages per block	64
Cleaning policy	Greedy	Page size	4 kB
Flash chip elements	8	Page read latency	0.025 ms
Number of channels	8	Page write latency	0.200 ms
Planes per element	8	Block erase latency	1.5 ms

[36]. In this paper, we simulated a NAND flash-based SSD. SSD specific parameter values used in the simulator are given in Table VI.

To conduct a fair performance evaluation of our proposed PGC algorithm, we fill the entire SSD with valid data prior to collecting performance information. Filling the entire SSD ensures that GC is triggered as new write requests arrive during our experiments. Specifically, for GC, we use a greedy algorithm that is designed to minimize the overhead of GC. The greedy algorithm selects a victim block to be erased whose number of valid pages is minimal. The more valid pages there are in the victim block, the longer it takes for GC to complete as the GC process needs to move more pages.

Our PGC algorithm can be applied to any existing GC schemes, such as idle-time or reactive. In the idle-time GC scheme, the GC process is triggered when there are no new incoming requests and all queued requests are already serviced. In the reactive scheme, GC is invoked based on the number of available free blocks, without regard to the incoming request status. If the number of available free blocks is less than the set threshold, then the GC process is triggered; otherwise, it continues servicing requests. The reactive GC scheme is the default in the MSR SSD simulator, and we use it as our baseline (non-PGC) GC scheme. The lower bound of the threshold in our simulations is set as the 5% of available free blocks. Ongoing GC is never preempted in the baseline GC scheme in our simulations. MSR SSD simulator implements a multichannel SSD, and GC operates per channel basis. In our experiments, even if one channel is busy for GC, any incoming requests to other channels can be serviced. The preemption occurs only if the incoming request is on the same channel where GC is running.

We use a mixture of real-world and synthetic traces to study the efficiency of our semi-PGC scheme. We use synthetic workloads with varying parameters such as request size, interarrival time of requests, read access probability, and sequentiality probability in access.² The default values of the parameters that we use in our experiments are shown in Table VII.

An exponential distribution and a Poisson distribution are used for varying request sizes and interarrival times of requests. Those distributions are well used to cover a variety of scenarios of workload cases in particular for the distribution of request arrivals. We vary one parameter while other parameters are fixed.

²If a request starts at the logical address immediately following the last address accessed by the previously generated request, we consider it a sequential request; otherwise, we classify it as a random request.

TABLE VII
DEFAULT PARAMETERS OF SYNTHETIC WORKLOADS

Parameter	Value
Request size	32 kB
Interarrival time	3 ms
Probability of sequential access	0.4
Probability of read access	0.4

TABLE VIII
CHARACTERISTICS OF REALISTIC WORKLOADS

Workload	Average Req. Size (kB)	Read (%)	Arrival Rate (IOP/s)	Bursty Write (%)
Financial [41]	7.09	18.92	47.19	0.14
Cello [38]	7.06	19.63	74.24	31.32
TPC-H [44]	31.62	91.83	172.73	11.29
OpenMail [17]	9.49	63.30	846.62	0.01

Note that bursty write percentage denotes the amount of write requests with less than 1.5 ms of interarrival times.

We use four commercial I/O traces, whose characteristics are given in Table VIII. We use write dominant I/O traces from an OLTP application running at a financial institution made available by the Storage Performance Council (SPC), referred to as the financial trace, and from Cello99, which is a disk access trace collected from a time-sharing server exhibiting significant writes which was running the HP-UX operating system at the Hewlett-Packard Laboratories. We also examine two read-dominant workloads. Of these two, TPC-H is a disk I/O trace collected from an OLAP application examining large volumes of data to execute complex database queries. Finally, a mail server I/O trace referred as OpenMail is evaluated.

While the device service time captures the overhead of GC, it does not include queuing delays for pending requests. Additionally, using an average service time does not capture response time variances. In this paper, we utilize the system service response time measured at the block device queue and the variance in response times. Our measurement captures the sum of the device service time and the additional time spent waiting for the device (queuing delay) to begin to service the request.

B. Performance Analysis of Semi-PGC

The following garbage collection schemes are evaluated in this section.

- 1) NPGC: A non-PGC scheme.
- 2) PGC: A semi-PGC scheme with both merging and pipelining enabled.

1) *Performance Analysis for Synthetic Workloads:* To evaluate the performance of PGC with various characteristics of input workloads, we start evaluating PGC with various synthetic workloads. GC may have to be performed while requests are arriving. Recall that GC is not preemptible in the baseline GC scheme and incoming requests during GC are delayed until the ongoing GC process is complete. Fig. 10 shows the performance improvements when enabling GC preemption.

a) *Request size:* Fig. 10(a) shows the improvements of performance and variance by PGC for different request sizes. In this experiment, we vary the request size as 8, 16, 32, and 64 kB. These values are chosen because the average request size of realistic workloads is between 7 and 31 kB, as given

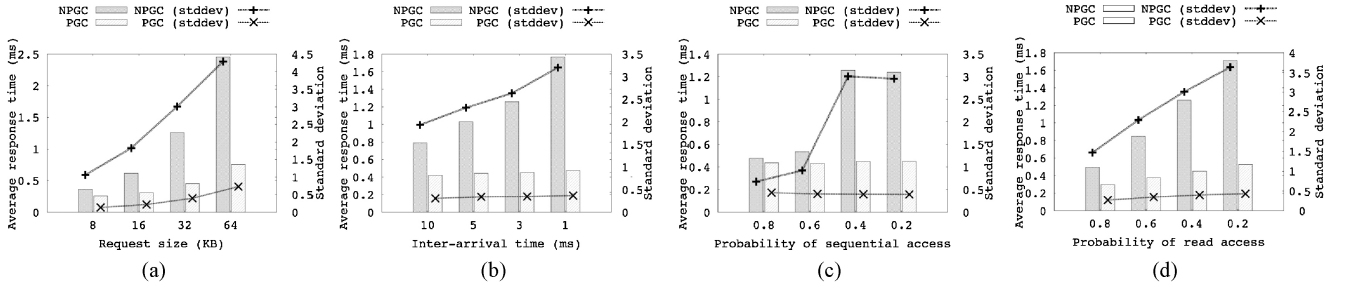


Fig. 10. Performance improvements of PGC for synthetic workloads. Average response times and standard deviations are shown with different parameters of synthetic workloads. (a) Request size. (b) Interarrival time. (c) Sequentiality. (d) Read ratio.

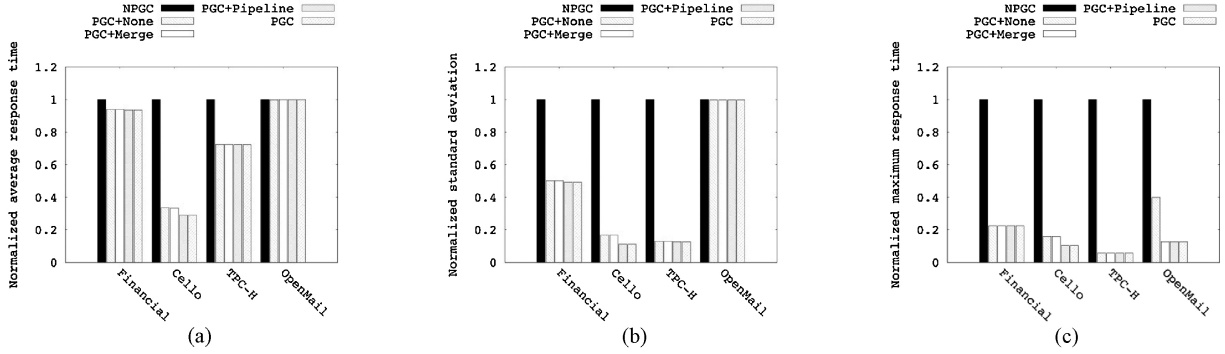


Fig. 11. Performance improvements of PGC and PGC+pipelining for realistic server workloads. (a) Average response time. (b) Variance of response time. (c) Maximum response time.

in Table VIII. For a small request size (8 kB), we see the improvement in response time by 29.44%. Furthermore, the variance of average response times decreases by 87.31%. As the request size increases, we see further improvements. For a large request (64 kB), the response time decreases by up to 69.21% while its variance decreases by 83.03%.

b) *I/O arrival rate*: Similar to the improvement with respect to varying request sizes, we also see an improvement with respect to varying the arrival rate of I/O requests. Typical response time of a request on a page is less than 1 ms without GC while it can be as high as 3–4 ms when the page request is queued up due to GC. Based on this observation, we vary the interarrival time between 1 and 10 ms in our experiments. In Fig. 10(b), it can be seen that PGC is minimally impacted by intense arrival rate. In contrast, the system response times and their variances for the baseline (NPGC) increase with respect to the request arrival rate.

c) *Sequential access*: Random workloads (where consecutive requests are not next to each other in terms of their access address) are known to be likely to increase the fragmentation of SSD, causing a GC overhead increase [21], [15]. We experiment with PGC and NPGC by varying the sequentiality of requests. Fig. 10(c) illustrates the results. As can be seen, NPGC exhibits a substantial increase in system response time and its variance for a 60% sequential workload while PGC performance levels remain constant for all levels of sequentiality.

d) *Write percentage*: Writes are slower than reads in SSDs because flash page writes are slower than reads (recall unit access latency for reads and writes, 25 μ s and 200 μ s, respectively) and GC can incur further delays. In Fig. 10(d), we see the improvement of PGC as the percentage

of writes within the workload increases. Overall, we observe that PGC exhibits a marginal increase in response time and variance compared to the NPGC scheme. For example, PGC performance slows down by only 1.77 times for an increase of writes in workloads (from 80% to 20% of reads) while NPGC slows down by 3.46 times.

From the performance analysis with synthetic workloads, we can observe a firm trend that PGC improves the performance, regardless of workload characteristics, and has a beneficial impact on the performance when the workload is heavier (e.g., larger request size, shorter interarrival time, less sequentially, and more write access).

2) *Performance Analysis for Realistic Server Workloads*: This section evaluates the performance of PGC with realistic server workloads. Merging and pipelining techniques and the safeguard are evaluated individually. The following GC schemes are added for the evaluation in this section.

- 1) PGC+None: A semi-PGC scheme without any optimization techniques.
- 2) PGC+Merge: Only merging technique enabled PGC.
- 3) PGC+Pipeline: Only pipelining technique enabled PGC.

Fig. 11 presents the improvement of system response time and variance over time for realistic workloads. For write-dominant workloads, we see an improvement in average response time by 6.05% and 66.56% for Financial and Cello, respectively [refer to Fig. 11(a)]. Fig. 11(b) shows a substantial improvement in the variance of response times. PGC reduces the performance variability by 49.82% and 83.30% for each of the workloads. In addition to the improvement in performance variance, we observe that PGC can further reduce the maximum response time of NPGC by 77.59% and 84.09% for Financial and Cello traces as illustrated in Fig. 11(c).

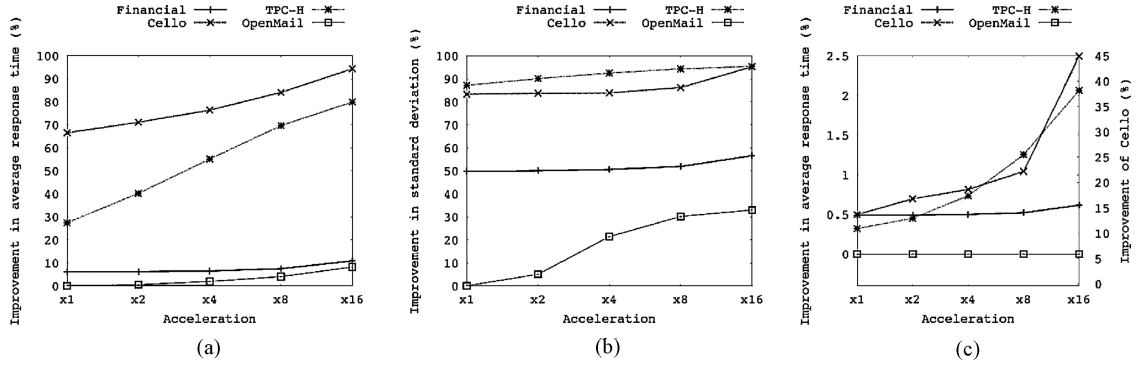


Fig. 12. Scalability tests by increasing the arrival rate of I/O requests. (a) Average response time. (b) Variance of response time. (c) Improvement in average response time of PGC and pipelining over PGC.

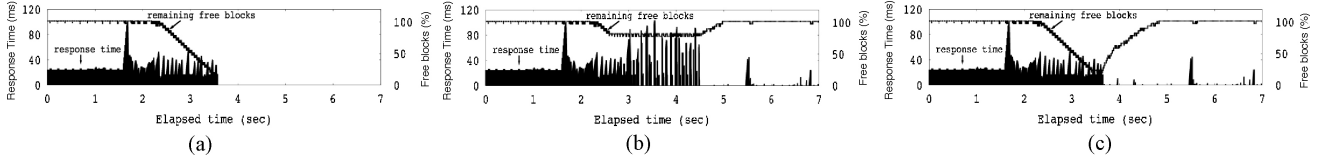


Fig. 13. Impact of hard threshold. The benchmark is Cello. (a) No hard threshold. (b) $T_{hard} = 80\%$ of T_{soft} . (c) $T_{hard} = 20\%$ of T_{soft} .

TABLE IX
PERCENTAGE OF NAND FLASH COMMANDS AFFECTED BY
MERGING AND PIPELINING

Benchmark	Merging		Pipelining	
	Read (%)	Write (%)	Read (%)	Write (%)
Financial	0.10	0.13	7.29	36.71
Cello	0.01	0.14	10.46	44.23
TPC-H	0.01	0.01	8.82	0.79
OpenMail	0.00	0.00	0.00	0.00

For the OpenMail trace PGC does not show a significant improvement for performance and variance, as we expected for read-dominant traces. However, PGC reduces the maximum response time by 60.26%. Interestingly for TPC-H, although it is a read dominant trace, we observe a substantial improvement for performance and variance. TPC-H is a database application. The disk trace includes a phase of application run that inserts tables into a database, which is shown as a series of large write requests (around 128 kB) for database insert operations.

Moreover, we observe further improvement by the pipelining technique on PGC in the Fig. 11.

Table IX shows how much the merging and pipelining contribute to the performance enhancement. The numbers shown in this table are the percentage of NAND flash commands affected by merging or pipelining among all flash commands issued by the incoming requests. Let N_w be the number of total write requests and N_r , the number of total read requests. The number of actual flash commands may not be the same because a request may span to multiple commands to multiple packages. Let us denote the number of write commands by C_w and that of read commands by C_r . Out of C_w commands, M_w commands are merged into commands issued by the ongoing GC. Similarly, P_w commands are pipelined with commands of GC. Then, the percentage of write commands affected by merging is computed by $M_w/(C_w + C_r)$. The percentage of write commands affected by pipelining is $P_w/(C_w + C_r)$. Those of read commands are computed in the same way.

It is shown in Table IX that the chance of merging is very low. Especially, the chance of merging and pipelining for OpenMail is less than 0.001%. However, we can still see that a high reduction of maximum response time can be achieved for OpenMail by I/O merge technique in Fig. 11, although the average performance is not improved significantly.

The chance of pipelining is higher than that of merging. For Cello, an improvement is observed in the average response time of PGC by 13.69% and its performance variance by 33.53%. Note that pipelining one command may not contribute to improving the performance because a request may span to multiple read or write commands.

Continuous GC preemption can cause starvation of free blocks. Thus, we develop a mechanism that can avoid a situation where an entire system becomes completely unserviceable because no free blocks are available. For this, we implement our PGC algorithm with a hard limit of available free blocks. Our algorithm now has two thresholds, one is for triggering the GC process and the other is for stopping preemption. Once the number of free blocks reaches T_{hard} , SSD stops GC preemption. A hard limit (T_{hard}) is set for a lower bound of the number of free blocks available in SSD.

To illustrate the effect of our extra threshold, we use an amplified Cello trace where the arrival rate of I/O requests are 16 times higher and the average request size of our test workload is about 300 kB. Cello is chosen because Cello is the most write-intensive workload among the four benchmarks, but with the original traces, we did not observe the shortage of free blocks incurred by preemption. To evaluate the impact of the safe guard, we had to amplify the trace artificially. In Fig. 13(a), we see the situation where there are no free blocks left due to continuous GC preemption and the SSD is not available to service the I/O requests. It captures a zoomed-in view of a region for 7 seconds of entire simulation run. The remaining free blocks indicate the ratio of the number of available free blocks over the minimum number of free blocks. The minimum number of free blocks corresponds to the soft

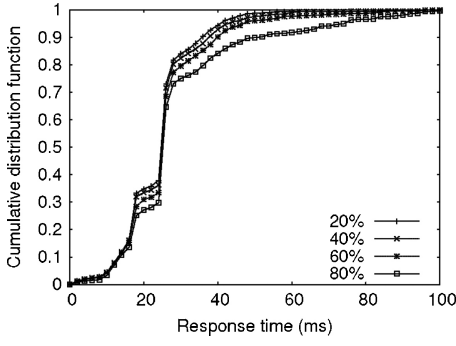


Fig. 14. Tradeoff between response time and hard limit. The benchmark is Cello.

threshold (T_{soft}) which is 5% of the total number of blocks as shown in Table VI. On the contrary, in Fig. 13(b) and (c), we see that the SSD handles the starvation of free blocks in the SSD by adjusting T_{hard} . We see that the lower T_{hard} shows better response time while it exhausts more free blocks.

Since there exists a tradeoff between the number of free blocks and response times, we evaluate the impact of performance in terms of response time according to T_{hard} . Fig. 14 shows the cumulative distribution function of response time for different T_{hard} . The average response times (in ms) are shown below each graph in the order of increasing the percentage of hard limit (T_{hard}). As we lower T_{hard} , we see overall response time improve. For example, we observe 18% improvement in average I/O response times when we lower T_{hard} from 80% to 20% of T_{soft} .

3) *Performance Sensitivity Analysis*: As shown in Fig. 12(a) and (b), with respect to increasing arrival rate, average response time and variance also improve. In particular, improvements in response times can be seen for write-dominant workloads (Financial and Cello) compared to read-dominant workloads in Fig. 12(a). For TPC-H, we see a gradual improvement for the performance variability. Overall, we observe that PGC can increase the performance and improve the variance up to 90% for a 16 times more bursty workload (i.e., the I/O arrival rate is increased by 16 times). Fig. 12(c) shows further improvements of the GC pipelining technique. In this figure, improvements in average response time for Cello can be clearly observed. Note that the scale for Cello is the right y-axis. For the other workloads, the benefit of the pipelining is not evident until the trace is accelerated significantly. The Financial and TPC-H exhibit a similar trend, but the OpenMail does not benefit from the pipelining because its chance is very low. However, we can still observe that the gaps of performance and variance are widened as the arrival rate of I/O requests increases. In other words, the GC pipelining technique makes PGC enabled SSDs robust enough to provide a sustained level of performance.

In addition to the greedy GC algorithm, we implemented two more GC algorithms to evaluate the performance of our proposed PGC for various real workloads. We implemented an Idle-based proactive GC algorithm where GC is triggered when an idle time is detected. For implementing idle time detection algorithm in workloads, we used a well-regarded heuristic online algorithm as in [13]. A wear-level aware GC

algorithm has also been implemented [19]. Unlike the greedy GC algorithm, wear-level aware GC algorithm considers the wear-levels of blocks to avoid selecting a block that has experienced more erase operations than the average wear-out. The wear-level aware GC algorithm aims to distribute erase operations evenly across blocks.

Fig. 15 shows the improvement of PGC against NPGC for various GC algorithms and various real workloads. We see that GC preemption works well regardless of GC algorithms. However, we see that the performance improvement of the idle-based algorithm is smaller than Fig. 11. It is because idle-based GC algorithm can run GC in background, which does not hurt the I/O service time. We also observe that Greedy-PGC outperforms idle-NPGC for all the traces except for OpenMail. Even though GC runs during idle times, GC still has to run upon write requests when they come in a bursty manner. In case of OpenMail, the average response time and standard deviation of the idle-based GC algorithm is slightly higher than those of the baseline greedy GC algorithm. We speculate that running GC during idle times could make the operation sequence different, which affects the results; however, this can be attributed to simulation artifact. Wear-aware GC algorithm does not show significant difference from the baseline of greedy GC algorithm.

From these experiments, we can observe that PGC reduces the response time and the variation regardless of GC algorithms. More importantly, it is shown that the PGC with a greedy GC algorithm (Greedy-PGC) that is triggered on demand will outperform the NPGC with a GC running during idle time (idle-NPGC) in the background.

All the preceding experiments in this section were done without write buffer. In this experiment, we study the impact of write buffer on SSD. We considered STT-RAM-based write buffer. The read and write latency of STT-RAM is 20 ns for both operations. STT-RAM has 10^{15} times of program/erase operation cycles, which is much higher than in NAND flash. Write-regulation technique that is a sort of selective write-buffering [23] can be employed if the lifetime of the STT-RAM buffer is seriously concerned. In our write-buffer implementation, data blocks are flushed into SSD whenever idle times in workloads are detected by flush operation.

Fig. 16 shows the improvement of the average response time by using PGC compared against NPGC when an 1 MB write buffer is employed. Compared with Fig. 11(a), the performance improvement by using PGC is decreased, but PGC still improves the performance by 0.47%, 27.74%, 11.97%, and 0.04% for Financial, Cello, TPC-H, and OpenMail, respectively. This experiments demonstrates that the proposed PGC improves the performance of write-intensive workloads even if a write buffer is employed.

C. F-PGC Evaluation

After extensive evaluation of the semi-PGC, we evaluate F-PGC and compare it with PGC. F-PGC has been evaluated with the same simulation environment described in Section V-A. We applied PGC and F-PGC to four realistic server workloads. We also implemented PGC+SE where suspend/resume commands are supported only for the erase op-

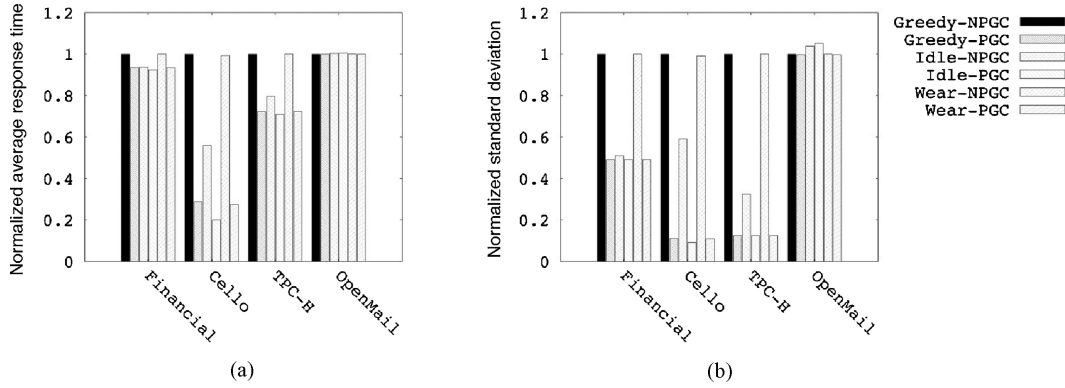


Fig. 15. Performance improvement of PGC for different GC algorithms. (a) Average response time. (b) Variance of response times.

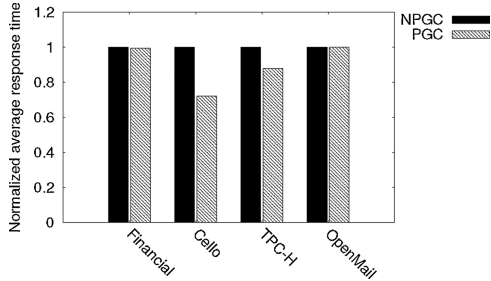


Fig. 16. Performance improvement of PGC over NPGC when an 1 MB write buffer is employed.

eration. Note that suspend/resume commands can be operable with read, write, and erase operations to implement F-PGC. The following GC schemes are evaluated in this section.

- 1) PGC: A semi-PGC scheme.
- 2) PGC+SE: PGC with suspend/resume commands being supported only for the erase command.
- 3) F-PGC: An F-PGC where suspend/resume commands are supported for read, write, and erase commands.

The suspend command takes up to $20\mu\text{s}$ [37] since a phase can last up to $20\mu\text{s}$. Therefore, we assume the overhead of suspending all the operations as $20\mu\text{s}$.

Fig. 17 shows the normalized average response time and the normalized variance of response times. As shown in Fig. 17(a) and (b), PGC+SE improves the average response time by up to 8.21% and the standard deviation by up to 29.63% compared to PGC. In case of F-PGC, it improves them by up to 68.13% and 83.59%, respectively. F-PGC shows significant improvements for Cello and TPC-H. Our conjecture is that Cello and TPC-H contain large amounts of bursty write requests, and F-PGC allows preemption on erase operations. Table VIII presents the percentages of write requests with less than 1.5 ms of interarrival time for workloads. Note that 1.5 ms is the block erase time on flash. Cello and TPC-H have significantly higher percentages of bursty write requests than Financial and OpenMail. If an erase operation is not preemptible (which does in F-PGC), request during the erase operation will be delayed. Though Financial and Cello are write dominant, Cello is bursty, while Financial is not bursty. Thus, F-PGC is not very effective for Financial. TPC-H is a read-dominant workload; however, most of bursty write requests are gathered in the first part of the workload (less

than 10% of total simulated time), and the remaining portion is mostly read requests, thus, F-PGC could significantly benefit from the first bursty write-dominant phase. OpenMail is read dominant, which has minimal impact on F-PGC.

The performance gain came mostly from preempting the erase and write operations. In our experiment, we allowed to preempt the read operation, but preempting the read operation did not have much impact on the performance because its chance for preemption was low and the latency of read was very short. Depending on the implementation, preempting the read operation may not be required.

VI. RELATED WORK

To offer predictable performance, real-time FTLs [10], [34] adopt a similar GC scheme where incoming requests are serviced while GC is running. They will need additional free blocks in order to buffer incoming write requests to avoid interruptions. When a block is full, it is queued to be cleaned later by the GC process. If any write requests come to that block, they will be directed to a temporary buffer until the block is cleaned, then the pages in the buffer are moved to the original block, or their role is switched. The proposed PGC and FPGC do not need an additional buffer because they exploit the page buffer that already exists in the flash memory device (as explained in Section III-A).

PGC is discussed in [7] as a possible method to meet the constraints of a real-time system equipped with NAND flash. They proposed creation of a GC task for each real-time task so that the corresponding GC task can prepare enough free blocks in advance. In a real-time environment both GC tasks and real-time tasks need to be preemptible. However, since NAND flash operations cannot be interrupted, these are defined as atomic operations. In contrast, our work provides a comprehensive study on the impact of the PGC in an SSD environment (compared to real-time environment) and we emphasize optimizing performance by exploiting the internal parallelism of the NAND flash device (e.g., the multiplane command and pipelining [32]).

Since it is well known that GC has significant adverse impact on the performance of SSD [10], [16], [25], [34], GC has attracted researchers' interest. Han [16] proposes using prediction to reduce the overhead of GC. An analytical model of the performance of GC [5] is developed to analyze the impact of GC on the performance. Recently, Wu [39] reported

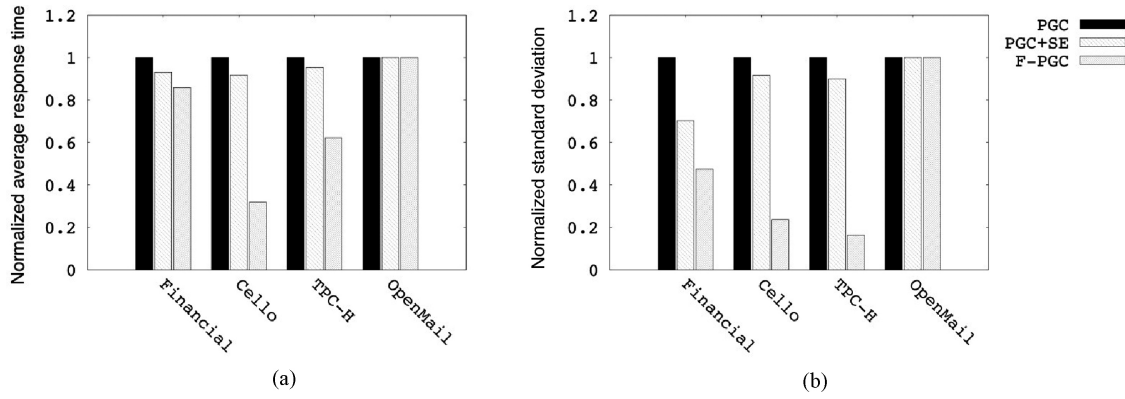


Fig. 17. Performance improvements of PGC+SE and F-PGC for realistic server workloads. (a) Average response time. (b) Variance of response times.

that suspending the write and erase operations help to improve the performance. Although GC is not considered in his paper, his observation is in full agreement with ours. Kim [25] proposes a coordinated GC mechanism for an array of SSDs to improve performance degradation due to GC incoordination of individual SSDs.

In the HDD domain, semipreemptible I/O has been evaluated [12] and its extension to RAID arrays also has been studied [12] by allowing preemption of ongoing I/O operations to service a higher priority request. To enable preemption, each HDD access operation (seek, rotation, and data transfer) is split into distinct operations. In-between these operations, a higher priority I/O operation can be inserted. In the case of PGC, we allow preemption of GC to service any incoming request. We split GC operations into distinct operations and insert incoming requests in between them. In addition, we provide further optimization techniques while inserting requests.

VII. CONCLUDING REMARKS

SSDs offer several advantages over HDDs: lower access latencies for random requests, lower power consumption, lack of noise, and higher robustness to vibrations and temperature. Although SSDs can offer better performance on average than HDDs in terms of I/O throughput (MB/s) or access latency, it often suffers from performance variability because of GC. From our empirical study, we observed that there are sudden throughput drops in COTS SSDs when increasing the percentage of writes in workloads. While GC is triggered to clean invalid pages to produce free space, incoming requests can be pending in the I/O queue, delaying their services until the GC finishes. This problem can become even more severe for bursty write-dominant workloads which can be observed in server-centric enterprise or HPC workloads.

To address this problem, we propose a semi-PGC that allows incoming requests to be serviced even before GC finishes by preempting ongoing GC. We identified preemption points that incur negligible overhead during GC and found four states that prevent GC from starvation of I/O service that can occur due to excessive preemption. We enhanced the performance even further by merging I/O requests with internal GC I/O requests and pipelining requests of the same type. We performed comprehensive experiments with synthetic and realistic traces. It is demonstrated by experiments that the proposed PGC can improve the average I/O response time by up to 66.56% and

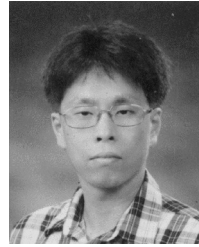
variance of response times by up to 83.30%. We applied PGC for accelerated workloads where interarrival time is shortened and evaluated with different GC schemes including idle-based proactive GC scheme and wear-aware selection algorithm. PGC exhibits significant performance improvement regardless of GC schemes for those workloads.

This paper also explored the feasibility of F-PGC. Assuming that there is a NAND flash memory that supports suspend/resume commands for read, write, and erase operations, we can implement F-PGC without incurring excessive overhead. Our evaluation result shows that F-PGC can further improve the average response time and the variation of response times by up to 14.57% and 52.48%, respectively, compared to PGC.

REFERENCES

- [1] N. Agrawal, V. Prabhakaran, T. Wobber, J. D. Davis, M. Manasse, and R. Panigrahy, "Design tradeoffs for SSD performance," in *Proc. Usenix Annu. Tech. Conf.*, Jun. 2008, pp. 57–70.
- [2] ARM. (2009). *ARM Security Technology* [Online]. Available: <http://infocenter.arm.com/>
- [3] J. Brewer and M. Gill, *Nonvolatile Memory Technologies With Emphasis on Flash (A Comprehensive Guide to Understanding and Using Flash Memory Devices)*. Hoboken, NJ: Wiley, 2008.
- [4] J. S. Bucy, J. Schindler, S. W. Schlosser, and G. R. Ganger. (2008). *The DiskSim Simulation Environment Version 4.0 Reference Manual* [Online]. Available: <http://www.pdl.cmu.edu/DiskSim/>
- [5] W. Bux and I. Iliadis, "Performance of greedy garbage collection in flash-based solid-state drives," *Perform. Eval.*, vol. 67, no. 11, pp. 1172–1186, Nov. 2010.
- [6] P. Carns, R. Latham, R. Ross, K. Iskra, S. Lang, and K. Riley, "24/7 characterization of petascale I/O workloads," in *Proc. Workshop Interfaces Archit. Scientific Data Storage*, 2009, pp. 1–10.
- [7] L.-P. Chang, T.-W. Kuo, and S.-W. Lo, "Real-time garbage collection for flash-memory storage systems of real-time embedded systems," *ACM Trans. Embedded Comput. Syst.*, vol. 3, no. 4, pp. 837–863, Nov. 2004.
- [8] Y.-H. Chang, J.-W. Hsieh, and T.-W. Kuo, "Endurance enhancement of flash-memory storage systems: An efficient static wear leveling design," in *Proc. 44th Annu. Conf. Design Automat.*, Jun. 2007, pp. 212–217.
- [9] F. Chen, D. A. Koufaty, and X. Zhang, "Understanding intrinsic characteristics and system implications of flash memory based solid state drives," in *Proc. 11th Int. Joint Conf. Meas. Modeling Comput. Syst.*, 2009, pp. 181–192.
- [10] S. Choudhuri and T. Givargis, "Deterministic service guarantees for NAND flash using partial block cleaning," in *Proc. 6th IEEE/ACM/IFIP Int. Conf. Hardware/Software Codesign Syst. Synthesis*, Oct. 2008, pp. 19–24.
- [11] T.-S. Chung, D.-J. Park, S. Park, D.-H. Lee, S.-W. Lee, and H.-J. Song, "System software for flash memory: A survey," in *Proc. Int. Conf. Embedded Ubiquitous Comput.*, Aug. 2006, pp. 394–404.
- [12] Z. Dimitrijevi, R. Rangaswami, and E. Chang, "Design and implementation of semi-preemptible IO," in *Proc. USENIX Conf. File Storage Technol.*, Mar. 2003, pp. 145–158.

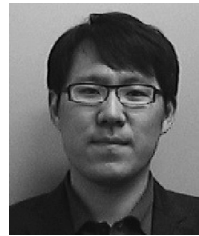
- [13] F. Douglass, P. Krishnan, and B. Marsh, "Thwarting the power-hungry disk," in *Proc. Winter USENIX Conf.*, 1994, pp. 293–306.
- [14] E. Gal and S. Toledo, "Algorithms and data structures for flash memories," *ACM Comput. Survey*, vol. 37, no. 2, pp. 138–163, 2005.
- [15] A. Gupta, Y. Kim, and B. Urgaonkar, "DFTL: A flash translation layer employing demand-based selective caching of page-level address mappings," in *Proc. 14th Int. Conf. Archit. Support Programming Languages Operating Syst.*, 2009, pp. 229–240.
- [16] L.-Z. Han, Y. Ryu, T.-S. Chung, M. Lee, and S. Hong, "An intelligent garbage collection algorithm for flash memory storages," in *Proc. 6th Int. Conf. Comput. Sci. Appl.*, part I, 2006, pp. 1019–1027.
- [17] Intel. *Intel Xeon Processor X5570 8M Cache, 2.93 GHz, 6.40 GT/s Intel QPI* [Online]. Available: <http://ark.intel.com/Product.aspx?id=37111>
- [18] Intel. *Intel X25-E Extreme 64GB SATA Solid-State Drive SLC* [Online]. Available: <http://www.intel.com/design/flash/nand/extreme/index.htm>
- [19] D. Jung, Y.-H. Chae, H. Jo, J.-S. Kim, and J. Lee, "A group-based wear-leveling algorithm for large-capacity flash memory storage systems," in *Proc. Int. Conf. Compilers, Archit., Synthesis Embedded Syst.*, 2007, pp. 160–164.
- [20] J.-U. Kang, H. Jo, J.-S. Kim, and J. Lee, "A superblock-based flash translation layer for NAND flash memory," in *Proc. 6th ACM IEEE Int. Conf. Embedded Software*, Oct. 2006, pp. 161–170.
- [21] H. Kim and S. Ahn, "BPLRU: A buffer management scheme for improving random writes in flash storage," in *Proc. USENIX Conf. File Storage Technol.*, Feb. 2008, pp. 1–14.
- [22] Y. Kim, R. Gunasekaran, G. M. Shipman, D. A. Dillow, Z. Zhang, and B. W. Settlemyer, "Workload characterization of a leadership class storage," in *Proc. 5th Petascale Data Storage Workshop*, Nov. 2010, pp. 1–5.
- [23] Y. Kim, A. Gupta, B. Urgaonkar, P. Berman, and A. Sivasubramaniam, "Hybridstore: A cost-efficient, high-performance storage system combining SSDs and HDDs," in *Proc. IEEE Int. Symp. Modeling, Anal. Simulation Comput. Telecommun. Syst.*, Jul. 2011, pp. 227–236.
- [24] Y. Kim, S. Gurumurthi, and A. Sivasubramaniam, "Understanding the performance-temperature interactions in disk I/O of server workloads," in *Proc. Int. Symp. High-Performance Comput. Archit.*, Feb. 2006, pp. 179–189.
- [25] Y. Kim, S. Oral, G. M. Shipman, J. Lee, D. A. Dillow, and F. Wang, "Harmonia: A globally coordinated garbage collector for arrays of solid-state drives," in *Proc. IEEE 27th Symp. Mass Storage Syst. Technol.*, May 2011, pp. 1–12.
- [26] J. Lee, Y. Kim, G. M. Shipman, S. Oral, F. Wang, and J. Kim, "A semi-preemptive garbage collector for solid state drives," in *Proc. IEEE Int. Symp. Performance Analysis Syst. Software*, Apr. 2011, pp. 12–21.
- [27] S.-W. Lee, D.-J. Park, T.-S. Chung, D.-H. Lee, S. Park, and H.-J. Song, "A log buffer-based flash translation layer using fully-associative sector translation," *ACM Trans. Embed. Comput. Syst.*, vol. 6, no. 3, p. 18, 2007.
- [28] S. Lee, D. Shin, Y.-J. Kim, and J. Kim, "LAST: Locality-aware sector translation for NAND flash memory-based storage systems," *SIGOPS Oper. Syst. Rev.*, vol. 42, no. 6, pp. 36–42, 2008.
- [29] H. Nijijima, "Design of a solid-state file using flash EEPROM," *IBM J. Res. Develop.*, vol. 39, no. 5, pp. 531–545, 1995.
- [30] ONFI. *Open NAND Flash Interface Specification* [Online]. Available: <http://www.onfi.org/>
- [31] S. Oral, F. Wang, D. A. Dillow, G. M. Shipman, and R. Miller, "Efficient object storage journaling in a distributed parallel file system," in *Proc. USENIX Conf. File Storage Technol.*, Feb. 2010, pp. 11–11.
- [32] S.-Y. Park, E. Seo, J.-Y. Shin, S. Maeng, and J. Lee, "Exploiting internal parallelism of flash-based SSDs," *Comput. Archit. Lett.*, vol. 9, no. 1, pp. 9–12, Jan.–Jun. 2010.
- [33] S. L. Pratt and D. A. Heger, "Workload dependent performance evaluation of the linux 2.6 I/O schedulers," in *Proc. Linux Symp.*, Jul. 2004, pp. 139–162.
- [34] Z. Qin, Y. Wang, D. Liu, and Z. Shao, "Real-time flash translation layer for NAND flash memory storage systems," in *Proc. Real-Time Embedded Technol. Appl. Symp.*, Apr. 2012, pp. 35–44.
- [35] M. Rosenblum and J. K. Ousterhout, "The design and implementation of a log-structured file system," *ACM Trans. Comput. Syst.*, vol. 10, no. 1, pp. 26–52, 1992.
- [36] J.-Y. Shin, Z.-L. Xia, N.-Y. Xu, R. Gao, X.-F. Cai, S. Maeng, and F.-H. Hsu, "FTL design exploration in reconfigurable high-performance SSD for server applications," in *Proc. 23rd Int. Conf. Supercomput.*, 2009, pp. 338–349.
- [37] Spansion. *Am29BL162C Data Sheet* [Online]. Available: <http://www.spansion.com/>
- [38] Super Talent. *Super Talent 128GB UltraDrive ME SATA-II 25 MLC* [Online]. Available: http://www.supertalent.com/products/ssd_detail.php?type=UltraDrive%20ME
- [39] G. Wu and X. He, "Reducing SSD read latency via NAND flash program and erase suspensions," in *Proc. 10th USENIX Conf. File Storage Technol.*, 2012.



Junghee Lee received the B.S. and M.S. degrees in computer engineering from Seoul National University, Seoul, Korea, in 2000 and 2003, respectively. He is currently pursuing the Ph.D. degree with the Georgia Institute of Technology, Atlanta.

From 2003 to 2008, he was with Samsung Electronics, Suwon, Korea, where he worked on electronic system level design of mobile systems-on-a-chip. His current research interests include architecture design of microprocessors, memory hierarchy, and storage systems for high performance computing

and embedded systems.



Youngjae Kim received the B.S. degree in computer science and engineering from Sogang University, Seoul, Korea, in 2001, the M.S. degree from the Korea Advanced Institute of Science and Technology, Daejeon, Korea, in 2003, and the Ph.D. degree in computer science and engineering from Pennsylvania State University, University Park, in 2009.

He is an I/O Systems Computational Scientist with the National Center for Computational Sciences, Oak Ridge National Laboratory, Oak Ridge, TN. He is currently an Adjunct Professor with the School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta. His current research interests include operating systems, parallel I/O and file systems, storage systems, emerging storage technologies, and performance evaluation.



Galen M. Shipman received the B.B.A. degree in finance and the M.S. degree in computer science from the University of New Mexico, Albuquerque, in 1998 and 2005, respectively.

He is currently a Data Systems Architect with the Computing and Computational Sciences Directorate, Oak Ridge National Laboratory (ORNL), Oak Ridge, TN. He is responsible for defining and maintaining an overarching strategy for data storage, data management, and data analysis spanning from research and development to integration, deployment,

and operations for high-performance and data-intensive computing initiatives at ORNL. Prior to joining ORNL, he was a Technical Staff Member with the Advanced Computing Laboratory, Los Alamos National Laboratory, Los Alamos, NM. His current research interests include high performance and data-intensive computing.



Sarp Oral received the M.Sc. degree in biomedical engineering from Cukurova University, Adana, Turkey, in 1996, and the Ph.D. degree in computer engineering from the University of Florida, Gainesville, in 2003.

He is currently a Research Scientist with the National Center for Computational Sciences, Oak Ridge National Laboratory, Oak Ridge, TN, where he is the Task Lead for the File and Storage Systems Research and Development Team with the Technology Integration Group. His current research interests

include performance evaluation, modeling, and benchmarking, parallel I/O and file and storage systems, high-performance computing and networking, computer architectures, fault tolerance, and storage technologies.



Jongman Kim received the B.S. degree in electrical engineering from Seoul National University, Seoul, Korea, in 1990, and the M.S. degree in electrical engineering and the Ph.D. degree in computer science and engineering both from Pennsylvania State University, University Park, in 2001 and 2007, respectively.

He is currently an Assistant Professor with the School of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta. His current research interests include hybrid multicore designs,

networks-on-chip, massively parallel processing architectures, and emerging memory systems. Before joining Pennsylvania State University, he was with LG Electronics, Seoul, Korea, and Neopoint, Inc., Ellisville, MO.